NLP501 · GROUP 2 · FINAL PROJECT

Agentic CV–JD ranking for Automation Test roles, under control

Sơn (problem & criteria) · Vũ (literature) · Đán (agentic HR, `coding` branch) · Hoa (data & metrics) · Thịnh (architecture & experiments)

1 · Truly agentic?

Where the agent decides, and what changes when it does.

2 · Data, processed well?

CV → facts, masking, chunking: measured against another team's generator truth and real CVs.

3 · AI output controlled?

13 written criteria: 7 checked in code on every output at run time, 6 measured offline.

Every number in this deck is generated by the repo (`docs/*_RESULTS*.md`) and reproducible with one command.

THE PROBLEM

Keywords can't see what matters in a tester's CV

- **Depth:** runs scripts (L1) → writes them (L2) → builds a framework (L3) → designs test architecture (L4).
- **Used vs listed:** a tool in *Skills* is a claim; a project that used it is evidence.
- **Hedges:** "familiar with Selenium", "currently learning Cypress".
- Keyword ATS rewards stuffing; an LLM judge is opaque, position-biased and can invent evidence.

Our score

$$\text{Score}(CV, JD) = \mathbb{1}[H(c)] \times \Sigma_k \, w(c,k) \cdot S(c,k)$$

H = knock-out on must-haves · w = company-type weights over 8 criteria · S = 0–100 per criterion

RQ	QUESTION
1	Structured extraction vs raw text?
2	HR-configured criteria fit the company?
3	Hybrid retrieval + reranking vs keyword / dense?
4	Agent vs fixed pipeline, same tools?
5	Errors by JD type and ambiguity?

POINT 2 · DATA

Three data sources, and no real candidate without consent

	OURS	CODING (ĐÁN)	REAL TEXT
Size	50 CVs × 12 JDs	200 CVs × 6 JDs	217 real, anonymised CVs
Built by	hand-written synthetic, built to stress the pipeline	generator with known latent facts; 37% Vietnamese; OCR-garbled CVs; fake PII	datasetmaster/resumes (MIT); its 4,600 Faker CVs excluded
Labels	silver (2 LLM annotators, κ 0.919) → human validation	generator rubric	none: NLP checks only
Role	development	held-out test	external validity

Pipeline (deterministic, 0 tokens)

normalise | **mask PII** | sections · dated roles | **structural chunks** | skills + hedge scope | years | depth from verbs | **CVProfile**

> **Masking is deletion, not instruction.** Asking an LLM to "ignore gender" is a hope; deleting it before indexing is a guarantee.

POINT 2 · MEASURED

Measuring against another team's data and scans found 10 bugs that tests had missed

VS THE CODING GENERATOR'S TRUTH	RESULT
Tool extraction F1 (clean CVs)	0.978 (0.994)
Years of experience, MAE	0.79 y
Automation depth exact	89%
Redaction leak, every PII field	0%
Garbled CVs flagged	7 / 7
Real CVs parsed · prose precision	217 / 217 · 90%
Scanned CVs via PaddleOCR (30 held out)	= text, 0% leak

- **Name leaked in 100%** of CVs headed by a bare name → **0%**
- Gender under an OCR-garbled label leaked
- Garbled-text flag caught 0 of 7 → 7 of 7
- Double spacing lost role titles: 8.0 → 14.8 automation years
- "pull requests" counted as the *requests* library (incl. our own trap CV)
- Jira counted as automation work for manual testers
- Gazetteer gaps: Azure DevOps, Kotlin, UFT/QTP, Sikuli, AutoIT, HP ALM
- Through OCR (no Vietnamese model): I read as "l" in tool names; headings and PII labels missing letters leaked the address on 15/15 Vietnamese scans → 0

POINT 2 · CHUNKING

Chunking matters little once a reranker reads it; turning CVs into facts matters a lot

NDCG@10	OURS	+ CE	CODING	+ CE
whole CV	**0.783**	0.818	**0.231**	0.620
fixed window	0.729	0.819	**0.231**	0.620
structural (per role, section-labelled)	**0.740**	**0.829**	**0.210**	**0.628**

With the cross-encoder, structural is best on both sets (+0.011 / +0.008, not significant). We keep it because only it labels each passage's section: the evidence checks need that.

+0.052

nDCG@10 on ours: rule scoring over the profile vs the same retrieval ($p \approx 0.001$)

+0.164

on the coding set (better on all 6 JDs)

RQ1: **facts beat text.** The gain comes from converting the CV into a structured profile, not from how it is chunked.

POINT 1 · AGENTIC

Rules where facts exist, the LLM where judgement is needed, humans where the decision is theirs

LOOP	DECISION	DECIDED BY	FIRED (OF 18 JDS)
L1 clarify	pause and ask HR, or proceed	rules first, then a typed LLM gate	3 (the vague ones)
L2 expand	widen search; which substitutes count	rule proposes, **HR approves**	3
L6 investigate	**which tool next on an unreadable CV**	**LLM chooses**, code verifies	**8 (36 CVs)**
L4 verify	retry a review whose claims fail checks	code (Q2–Q5)	9
L3 arbiter	swap a near-tie	LLM, both orders, $\|\Delta\| < 0.05$ only	13
L5 HR review	approve, re-weight, decide	**HR**; weights learned from decisions	—

8

distinct paths through the graph over 18 JDs (full agent)

1

path for the fixed pipeline with the same tools

12

nodes, 6 bounded loops, 3 human interrupts

0

scores set by the LLM (since v3)

POINT 1 · THE INVESTIGATOR (N12)

A tool-using loop for CVs the parser cannot read, which may not invent anything

Clean text: tool F1 **0.994**. OCR noise ("Us#d S%lenium W#bDriver"): **0.192**, so good candidates are knocked out by the data.

trigger | garbled, or must-have almost spelt

LLM picks | read_section | search (fuzzy) | propose(skill, quote) | finish

code verifies | quote in CV | names the skill | not hedged

≤ 4 CVs × 6 actions · repeated actions refused · stops when every missing must-have is decided · invalid output → knock-out stands

42 PLANTED CASES (QWEN3:8B)	RESULT
misspelt tools recovered	4 / 4
negatives kept out (lacking / only learning)	13 / 13
false recoveries	**0**
Sonnet 5.5 vs qwen3:8b, true facts from garbled CVs	5 vs 3

> A stronger model recovers more; **neither** recovers anything false, because code, not the model, decides what becomes a fact.

POINT 3 · CONTROL

Every LLM output is checked in code at run time, and every check has an action on failure

1. **The LLM never sets the score.** Rules over the parsed CV do.
2. **Typed in, canonical out.** Schemas; closed choice sets.
3. **Every claim is checked against the CV by code**: quotes exist and prove the right thing; no tool or years the CV doesn't show; no protected attributes.
4. **Every failure has a defined action**: retry with the exact failure, then drop or label.
5. **Humans own the decision**: clarification, substitutes, approval, audit log.
6. **The controls are measured too**, and no LLM judge is trusted before agreeing with humans.

Q	CRITERION	ACTION
Q2	quotes grounded	retry with failing quotes
Q4	claims match the parsed CV	retry, then [unverified]
Q5	no protected attributes	sentence removed
Q6	report claims supported	regenerate, then drop
Q7	arbiter order-stable	keep rule order
Q8	decisions right	safe default
Q11	blind to protected attributes	release blocker
Q13	LLM judge validated	not ground truth

13 criteria in docs/AI_OUTPUT_CRITERIA.md

POINT 3 · MEASURED

The scorecard of the full agent on the 18 benchmark JDs

100%

of 909 evidence quotes grounded in the CV (Q2)

2.1→0.6%

rationales contradicting the CV, before → after the verify loop (Q4); 2.9% without it

0

unsupported report claims shown to HR: 5.9% caught, 1.2% left after a retry and dropped (Q6)

100%

17 / 17 benchmark near-tie verdicts stable when the order is swapped (Q7)

Fairness (Q11): counterfactual twins

Each CV re-written with only a **name of the other gender**, a **gender**, **age** or **marital-status** line changed:

> **No score changed** in 6 twin variants × 1,800 (CV, JD) pairs; masked input identical in every twin.

Decisions (Q8)

- Ask-HR gate on Claude: asked on exactly the **3 of 18** JDs that need it, none of the others

POINT 3 · THE SURPRISE

A stronger model needed more control, not less

NDCG@10 (OURS / CODING)	JD READ BY QWEN3:8B	BY CLAUDE SONNET
LLM-only JD parser	0.884 / 0.824	**0.842 / 0.795**
Hybrid parser (rules first, LLM fills gaps)	**0.876 / 0.791**	**0.881 / 0.792**

- Sonnet reads JDs **more strictly**: "Jenkins or GitLab CI" became two hard must-haves (JD-008: 5 must-haves vs qwen's 2).
- Knock-out misses rose from **9 to 17**.
- The deterministic JD parser keeps the reading literal; the hybrid is stable under both models.

> **Lesson:** a better model is not a better system by default. Its extra capability showed up as extra confidence in a stricter reading, and only a control layer made the system robust to it.

Hybrid became the default by the decision rule we wrote before running: not worse by > 0.01 on either benchmark, better on one.

RESULTS · THE TEAM'S FROZEN BENCHMARK

NLP501-bench v1: which version of the project is best?

TEST SPLIT, NDCG@10	10 JDS	OURS (6)	ĐÁN'S (4)
B0 BM25	0.695	0.800	0.536
Structural chunks + CE	0.726	0.830	0.570
Đán's core (coding branch)	0.810	0.773	**0.866**
Our v2	0.751	0.806	0.669
Merged v3.1, deterministic	0.837	0.890	0.758
Merged v3.1, fixed pipeline (B4)	**0.848**	**0.890**	**0.784**
Merged v3.1, full agent (P6)	**0.847**	**0.897**	**0.772**

18 JDs from both datasets, 10 locked for test · 268 inputs pinned by SHA-256 · tuning on dev only

- **The merge is the best version measured**: +0.086 over our v2 ($p = 0.004$), +0.037 over Đán's core ($p = 0.22$, not yet significant).
- **Đán's core still wins on his own JDs** (0.866 vs 0.772); only the merge is strong on both.
- Facts beat text: +0.11 over the best retrieval-only system.

> One benchmark, agreed by the whole team, ends "each system wins on its own data".

RESULTS · RQ4

The agent doesn't add ranking points; it makes the system trustworthy

Test split: P6 0.847 vs B4 0.848 (p = 0.47). Ablations below on our 12 JDs.

SYSTEM	NDCG@10	Δ VS P6	PATHS	CONTRADICTED CLAIMS
P6 full agent	**0.882**	**—**	**5**	**0.9%**
B4 fixed pipeline	0.879	−0.003	1	4.0%
A1 – verify	0.883	+0.001	3	2.8%
A2 – company profile	0.878	−0.004	5	1.3%
A3 – arbiter	0.880	−0.002	5	0.9%
A4 – investigator	0.883	+0.001	5	0.9%

All ranking differences ≤ 0.004, none significant (12 JDs). A1–A4 ran before the two fixes on slide 13, which move our pool by ≤ 0.003.

- Rules set the score, so ranking is equal by design.
- **Verify** is what removes contradicted claims.
- **The arbiter** checks both orders and now judges only true near-ties.
- **The investigator** costs nothing on clean CVs and is the only part that can rescue a good CV from bad text. On the benchmark it rescued none, and was right to for 36 of 37 knock-outs.
- **Asking HR** pays where JDs are vague: on vague twins of all 18 JDs it recovers 57% of the lost nDCG@10 (+0.129, $p < 0.001$), 0.704 vs 0.626 for a pipeline that cannot ask.
- Price of control: about 2× the LLM calls.

WHAT THE BENCHMARK EXPOSED

A second dataset found two faults our own data never could

1 · A recall funnel sized for 50 CVs

Search carried the top 40 retrieval hits into screening: 80% of our pool, **20%** of Đán's 200. Good CVs never reached scoring.

Fix: rules are free, so the top 40 by rule score enter too. Decided on dev (+0.023 / −0.003).

0.60 → 0.77

full agent on Đán's test JDs (fixed pipeline 0.61 → 0.78; bank_api_qa 0.43 → 0.84)

2 · An arbiter beyond its mandate

A real trace: "Near-tie (Δ = −0.150) resolved by arbiter". With fused ordering, a negative gap passed the tie check, and the LLM lifted a 0.77 candidate over a 0.92 one.

Fix: |Δ| < 0.05, with a regression test.

90% → 100%

arbiter verdicts stable under an order swap

Both found by reading results and traces, not by unit tests (123 pass). Disclosed in the report: test was re-reported after fixes decided on dev.

ERRORS AND LIMITS

Where it fails, and what we can't claim yet

Test split: 10 avoidable misses, 2 false alarms (10 JDs)

- **All 6 errors on Đán's set** are on one vague JD; 4 are shared with Đán's core
- **3 of 5 on ours** are vocabulary: strong CVs that describe tools without naming them ("build servers"). Accepting those as weaker evidence left dev nDCG@10 unchanged: not adopted
- Đán's core on the same JDs: 14 misses, 1 false alarm
- the one false alarm: a must-have only listed, never used

Limitations

- open **Silver labels** from two LLMs. A third blind Claude agrees (κ 0.89): stable within one model family. Two teammates grade 120 blind pairs; κ ≥ 0.8 keeps them
- open **Synthetic ranking data**: consented real tester CVs being collected
- 12 + 6 JDs: differences < 0.02 are not significant
- Claude served through the Claude Code CLI as a stand-in for the API: behaviour carries over, latency and cost do not

TAKEAWAYS

What we would tell the next team building an agentic ranker

1 · Agentic where it earns it

Let the model decide what rules cannot: when to ask, how to read noisy text, which near-tie flips. Measure it by the paths it takes and the errors it prevents, not only by nDCG.

2 · Data work is measurable work

Measuring against another team's data and scanned CVs found 10 bugs, including a 100% name leak, that 100+ unit tests missed. Mask by deleting; structure CVs into facts.

3 · Control scales with capability

Write the criteria, check them in code, give every failure an action. The stronger model needed the control layer most.

Repo: feature/merged-pipeline · report: docs/FINAL_REPORT.md · benchmark: docs/BENCHMARK.md · criteria: docs/AI_OUTPUT_CRITERIA.md · 123 tests